Dokumentácia

Porovnanie dátových štruktúr

Hash tabuľka (reťazenie), Hash tabuľka (dvojité rozptýlenie), Červeno čierny strom a 2-3 strom

Miroslav Repický
2025/2026

1. Úvod

Táto dokumentácia popisuje implementáciu a experimentálne porovnanie štyroch dátových štruktúr určených na ukladanie a vyhľadávanie celých čísel:

- Hash tabuľka s reťazením (hash_chain) — kolízie sú riešené spájanými zoznamami v každom slote.
- Hash tabuľka s dvojitým rozptýlením (hash_double) — otvorené adresovanie s druhou hash funkciou pre výpočet kroku pri kolízii.
- Červeno-čierny strom (rbt) — samo vyvažujúci binárny vyhľadávací strom s garantovanou logaritmickou hĺbkou.
- 2-3 strom (t23) — vyvažujúci strom kde každý uzol má 2 alebo 3 deti, všetky listy sú na rovnakej úrovni.

Každá štruktúra podporuje tri základné operácie: vloženie kľúča (insert), vyhľadanie kľúča (search) a vymazanie kľúča (delete). Cieľom testovania bolo zmerať priemerný čas jednej operácie v závislosti od veľkosti štruktúry n a identifikovať, v akých situáciách je ktorá implementácia výhodnejšia.

2. Popis implementovaných dátových štruktúr

2.1 Hash tabuľka s reťazením (hash_chain)

Hash tabuľka s reťazením rieši kolízie tak, že každý slot tabuľky obsahuje spájaný zoznam prvkov, ktoré sa zahashujú do toho istého slotu. Pri vkladaní sa kľúč pridá na začiatok príslušného zoznamu, pri vyhľadávaní sa prechádza zoznam daného slotu.

Parametre rehashingu: Tabuľka sa rozšíri keď load faktor prekročí ALPHA_THRESHOLD = 0.75 (75 % zaplnenosť). Po rozšírení je nová spodná hranica LOWER_ALPHA = 0.20. Počiatočná kapacita je 16 slotov.

Dôsledok pre merania: Scenáre S1, S2, S7 (insert) a S5 (delete) merajú čas vrátane prípadného rehashingu/resizingu tabuľky. Čas rehashingu je O(n) jednorazovo, čo sa prejaví ako občasné zdvihy v krivkách najmä pri malých n.

Teoretická časová zložitosť: O(1) amortizovane pre insert, search aj delete pri rovnomernom rozložení kľúčov. V najhoršom prípade (všetky kľúče v jednom slote) O(n).

2.2 Hash tabuľka s dvojitým rozptýlením (hash_double)

Pri kolízii sa použije druhá hash funkcia na výpočet kroku, o ktorý sa posúvame v tabuľke pri hľadaní voľného slotu. Všetky prvky sú uložené priamo v poli.

Parametre rehashingu: Tabuľka sa rozšíri keď load faktor prekročí ALPHA_THRESHOLD_DOUBLE = 0.50 (50 % zaplnenosť). Spodná hranica po zmenšení: LOWER_ALPHA_DOUBLE = 0.20. Nižší prah oproti reťazeniu predchádza clustering — pri otvorenom adresovaní degraduje výkon rapídne nad load faktorom 0.7.

Dôsledok pre merania: Rovnako ako pri reťazení, S1/S2/S5/S7 merajú čas vrátane rehashingu. Vďaka nižšiemu prahu (0.5 vs 0.75) dochádza k rehashingu častejšie, no každý jednotlivý rehash je lacnejší (tabuľka je menej zaplnená).

Teoretická časová zložitosť: O(1) amortizovane pre všetky operácie. Citlivé na load faktor a preto je prah nastavený konzervatívne na 0.5.

2.3 Červeno-čierny strom (rbt)

Červeno-čierny strom je samovyvažujúci binárny vyhľadávací strom, kde každý uzol má priradenú farbu (červená/čierna).

Dôsledok pre merania: Na rozdiel od hash tabuliek neexistuje žiadny rehashing — čas operácie závisí výlučne od hĺbky stromu. Insert a delete obsahujú rotácie a prefarbenia uzlov, ktoré sú však O(log n) a teda zahrnuté v nameranom čase.

Teoretická časová zložitosť: O(log n) pre insert, search aj delete v najhoršom aj priemernom prípade.

2.4 2-3 strom (t23)

2-3 strom je vyvažujúci strom, kde každý interný uzol má buď 2 alebo 3 deti. Všetky listy sú na rovnakej úrovni — strom je dokonale vyvážený po výške. Vkladanie a mazanie udržiavajú túto vlastnosť štiepením a spájaním uzlov.

Dôsledok pre merania: Štiepenie a spájanie uzlov pri inserte/delete je súčasťou nameraného času. Vďaka vetviacemu faktoru 2–3 je strom nižší ako binárny strom , no réžia spracovania uzlov s dvoma kľúčmi je vyššia.

Teoretická časová zložitosť: O(log n) pre insert, search aj delete.

3. Metodológia testovania

3.1 Testovacie prostredie

Benchmark bol implementovaný v jazyku C++ (štandard C++17), kompilovaný s optimalizáciou -O2. Meranie času prebieha pomocou std::chrono::high_resolution_clock s presnosťou na nanosekundy. Výsledkom je priemerný čas jednej operácie (ns/op) spolu so štandardnou odchýlkou zo všetkých opakovaní.

Kompilácia a spustenie testovania v termináli:

```
>> g++ -O2 -std=c++17 -o benchmark benchmark.cpp hash_chaining.c hash_double.c rbt.c two_three_tree.c
>> ./benchmark
```

3.2 Testované veľkosti n

n (počet prvkov)
10 000
25 000
50 000
100 000
250 000
500 000
750 000
1 000 000
2 500 000
5 000 000

3.3 Typy scenárov

Scenáre S1–S7 (hromadné meranie, 4 opakovania): Meria sa celkový čas n operácií vydelený n. Pred každým opakovaním sa štruktúra znovu vytvorí a naplní (fáza setup, ktorá sa nemeria). Pre insert scenáre (S1, S2, S7) a delete scenár (S5) je súčasťou meraného času aj prípadný rehashing hash tabuliek.

Scenáre SC (čistá zložitosť, 50 opakovaní, blok 1 000 operácií): Meria sa blok 1 000 operácií na štruktúre veľkosti n, výsledok je čas/1 000. Štruktúra má počas merania veľkosť ~n. Cieľom je empiricky overiť časovú zložitosť sledovaním trendu ns/op vs n.

3.4 Interpretácia grafov

Všetky grafy používajú logaritmickú škálu na oboch osiach (log-log). Na takomto grafe:

- Konštantná zložitosť O(1) sa prejaví ako vodorovná priamka.
- Logaritmická zložitosť O(log n) sa prejaví ako pomaly rastúca krivka.

- Lineárna zložitosť O(n) by sa prejavila ako priamka so smernicou 1.

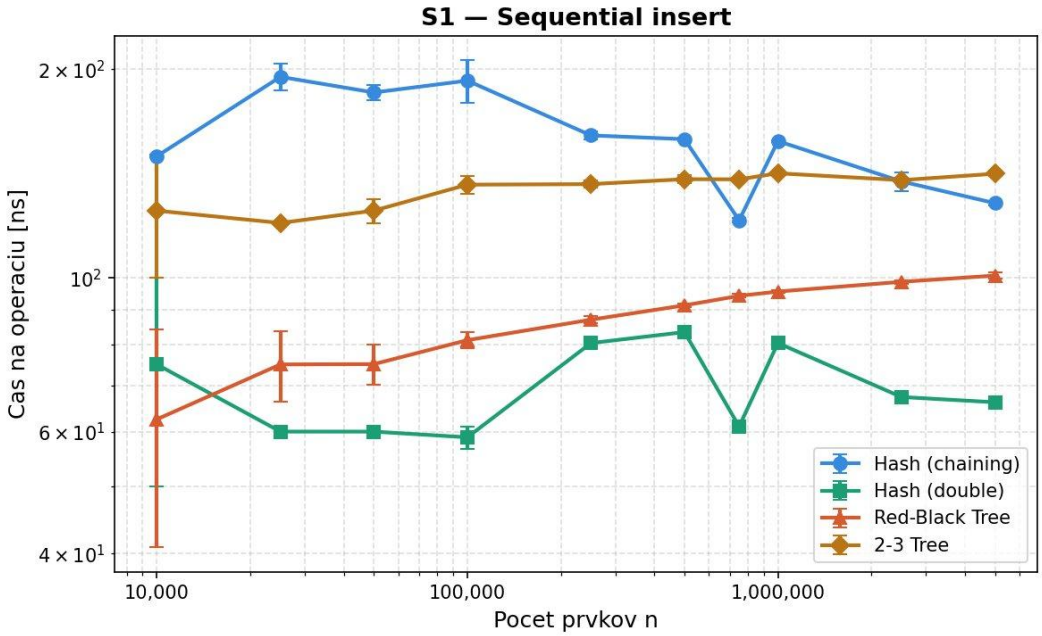
Chybové úsečky v grafoch (error bars) predstavujú štandardnú odchýlku z opakovaní. Väčšia odchýlka pri SC scenároch je očakávaná — meranie krátkych operácií (10–200 ns) je citlivejšie na šum operačného systému.

4. Výsledky testovacích scenárov (S1–S7)

S1 Sekvenčné vkladanie

Scenár meria rýchlosť vkladania kľúčov 1, 2, 3, …, n v poradí do prázdnej štruktúry.

Setup: prázdna štruktúra. Meria sa: *vloženie n sekvenčných kľúčov vrátane rehashingu hash tabuliek.*



Graf S1 — Sekvenčné vkladanie: čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Hash (dvojité) dosahuje najlepší výsledok pri sekvenčnom vkladaní. Pohybuje sa stabilne v rozsahu 60–80 ns/op naprieč všetkými veľkosťami n. Sekvenčné kľúče sa dobre distribuujú vďaka hash funkcii, čím sa minimalizujú kolízie.

Hash (reťazenie) vykazuje vyšší a kolísavejší čas (120–195 ns/op). Vyšší prah rehashingu (0.75 vs 0.50) znamená, že tabuľka je pri rehashingu plnšia a rehashing trvá dlhšie.

Red-Black strom rastie pomaly a plynulo (60–100 ns/op), a teda logaritmický charakter je zrejmý.

2-3 strom je konštantne v rozsahu 120–140 ns/op. Napriek dokonalému vyváženiu je pomalší ako RBT, čo naznačuje vyššiu réžiu spracovania uzlov s dvoma kľúčmi.

Štruktúra	n = 100 000	n = 1 000 000	n = 5 000 000
Hash (reťazenie)	192.5 ns	157.5 ns	128.2 ns
Hash (dvojité)	58.9 ns	80.5 ns	66.1 ns
Red-Black strom	81.3 ns	95.5 ns	100.8 ns
2-3 strom	136.3 ns	141.5 ns	141.3 ns

S2 Náhodné vkladanie

Vkladanie n náhodných unikátnych kľúčov z rozsahu 1 až 10n do prázdnej štruktúry. Simuluje realistické zaťaženie.

Setup: prázdna štruktúra. Meria sa: *vloženie n náhodných kľúčov vrátane rehashingu hash tabuliek.*

Graf S2 — Náhodné vkladanie: čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Kľúčový rozdiel oproti S1: náhodné kľúče spôsobujú cache missy pri stromoch, pretože pohyb po strome je nepredvídateľný. Pri n = 5 000 000 sú stromy 3–5× pomalšie ako hash tabuľky.

Hash (dvojité rozptýlenie) zostáva rýchle (55–140 ns/op) vďaka cache-friendly poli. Hash (reťazenie) je stabilné okolo 150–210 ns/op. Každý insert môže siahnúť do pamäte kvôli alokácii uzla zoznamu.

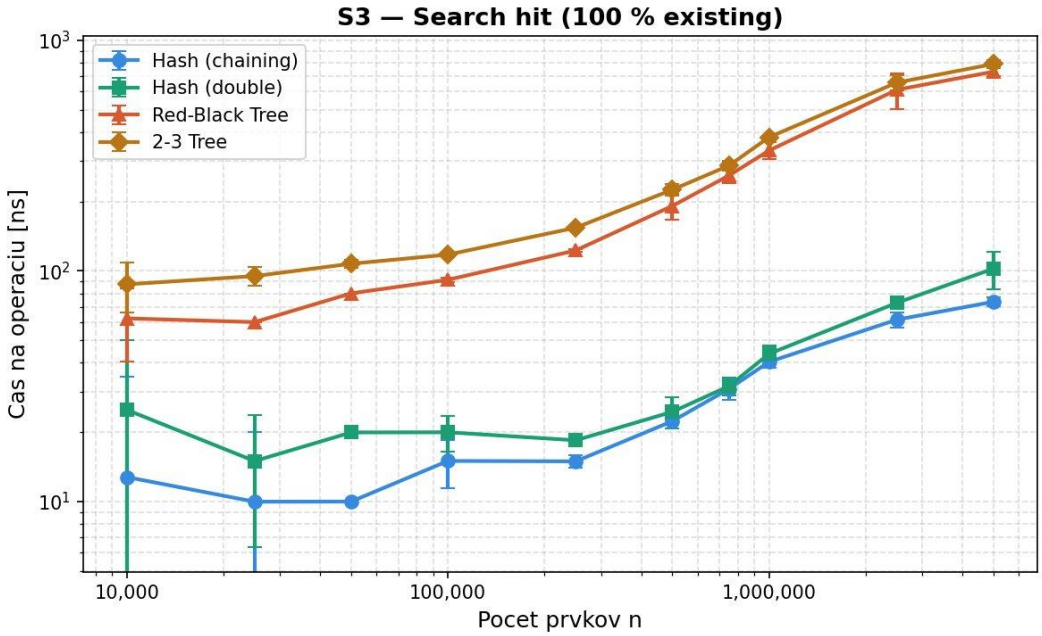
Stromy rastú jednoznačne logaritmicky: pri zdesaťnásobení n narastie čas o ~1.5×, čo zodpovedá O(log n). Pri n = 5 000 000 dosahujú RBT 654 ns/op a 2-3 strom 692 ns/op.

Štruktúra	n = 100 000	n = 1 000 000	n = 5 000 000
Hash (reťazenie)	176.1 ns	166.0 ns	210.9 ns
Hash (dvojité)	66.3 ns	104.5 ns	138.9 ns
Red-Black strom	170.0 ns	313.3 ns	654.8 ns
2-3 strom	181.3 ns	332.5 ns	692.1 ns

S3 Vyhľadávanie existujúcich kľúčov (100 % zhoda)

Vyhľadávanie kľúčov, ktoré v štruktúre existujú. Kľúče sú pred vyhľadávaním zamiešané, aby sa eliminoval efekt poradia.

Setup: vloženie n náhodných kľúčov. Meria sa: *vyhľadanie tých istých n kľúčov v náhodnom poradí. Rehashing nie je súčasťou merania.*



Graf S3 — Vyhľadávanie (hit): čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Toto je scenár, kde hash tabuľky dominujú najvýraznejšie. Hash (reťazenie) dosahuje pri n = 100 000 len 15 ns/op a pri n = 5 000 000 len 73 ns/op — rast je mierny a zodpovedá efektu cache miss pri väčších tabuľkách, nie algoritmickej zložitosti.

Stromy rastú výrazne logaritmicky: RBT ide z 91 ns/op pri n = 100 000 na 731 ns/op pri n = 5 000 000. Nárast o ~8×. 2-3 strom je konzistentne o ~8 % pomalší ako RBT, napriek teoreticky nižšej hĺbke stromu.

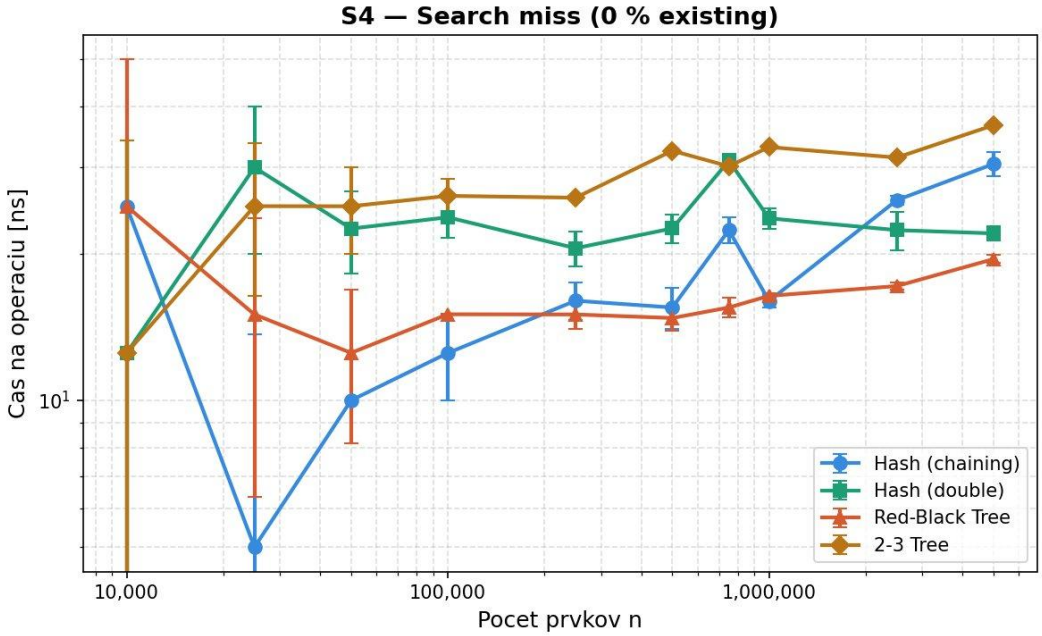
Hash (dvojité rozptýlenie) je mierne pomalší ako reťazenie pri veľkých n (102 vs 73 ns/op pri n = 5M) — pri vyhľadávaní môže dvojité hashovanie vyžadovať viac krokov ak má tabuľka vyššiu zaplnenosť v okolí hľadaného slotu.

Štruktúra	n = 100 000	n = 1 000 000	n = 5 000 000
Hash (reťazenie)	15.0 ns	40.4 ns	73.5 ns
Hash (dvojité)	20.0 ns	43.7 ns	102.4 ns
Red-Black strom	91.6 ns	332.8 ns	731.2 ns
2-3 strom	117.7 ns	380.1 ns	789.7 ns

S4 Vyhľadávanie neexistujúcich kľúčov (0 % zhoda)

Vyhľadávanie záporných kľúčov (−1, −2, …, −n), ktoré v štruktúre nikdy neboli. Záporné hodnoty sú garantovane mimo množiny vložených (kladných) kľúčov.

Setup: vloženie n kladných kľúčov. Meria sa: *vyhľadanie n záporných kľúčov. Rehashing nie je súčasťou merania.*



Graf S4 — Vyhľadávanie (miss): čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Všetky štruktúry sú pri search miss výrazne rýchlejšie ako pri search hit — a to z metodologického dôvodu, nie algoritmického. Záporné kľúče sú menšie ako všetky vložené (kladné) kľúče, čo spôsobuje:

- Hash tabuľky: záporný kľúč sa zahashuje do iného slotu ako kladné kľúče. Slot môže byť prázdny, čo znamená okamžité zamietnutie bez prechádzania zoznamu/probe sekvencie.
- Stromy: záporný kľúč je vždy menší ako koreň, a teda algoritmus ide vždy doľava až narazí na null. Ide deterministickou, cache-friendly cestou po ľavej hranici stromu, bez porovnávania vo vnútri.

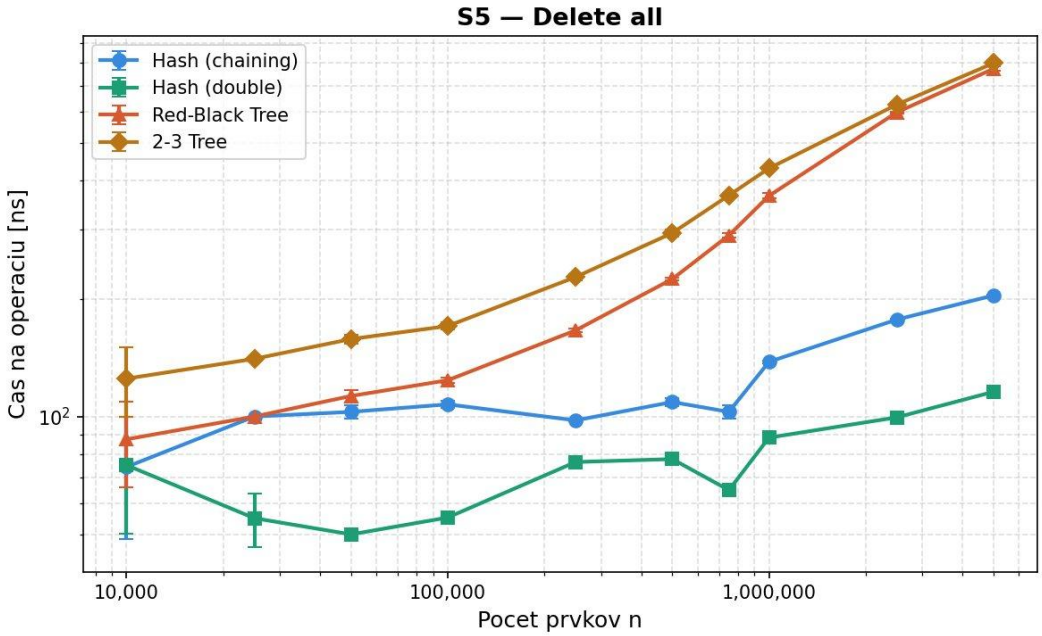
Preto S4 nemeria skutočný worst-case search miss. Pre realistický miss (kľúč v rozsahu vložených hodnôt, ale chýbajúci) by boli časy vyššie. RBT je v tomto scenári prekvapivo rýchly (~15–19 ns/op) a takmer konštantný. Ľavá hranica stromu má fixnú hĺbku bez ohľadu na n.

Štruktúra	n = 100 000	n = 1 000 000	n = 5 000 000
Hash (reťazenie)	12.5 ns	16.0 ns	30.5 ns
Hash (dvojité)	23.7 ns	23.6 ns	22.0 ns
Red-Black strom	15.0 ns	16.4 ns	19.5 ns
2-3 strom	26.3 ns	33.1 ns	36.7 ns

S5 Vymazanie všetkých prvkov

Vymazanie všetkých n prvkov z naplnenej štruktúry v náhodnom poradí. Testuje implementáciu mazania vrátane rebalancingu stromov a resizingu hash tabuliek.

Setup: vloženie n náhodných kľúčov. Meria sa: *vymazanie všetkých n kľúčov v náhodnom poradí vrátane prípadného resizingu hash tabuliek.*



Graf S5 — Mazanie všetkých prvkov: čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Mazanie je pre stromy najnákladnejšia operácia, pretože vyžaduje rebalancing (rotácie pri RBT, štiepenie/spájanie uzlov pri 2-3 strome) po každom mazaní. Pri n = 5 000 000 dosahuje RBT 773 ns/op a 2-3 strom 799 ns/op.

Hash (dvojité rozptýlenie) je jednoznačne najrýchlejšie (50–115 ns/op): mazanie sa označí tombstone značkou, čo je O(1) bez akejkoľvek réžie navyše. Resizing (zmenšenie tabuľky pri LOWER_ALPHA = 0.20) sa prejaví ako výchylky pri menšom n.

Hash (reťazenie) je pomalšie (74–203 ns/op) — mazanie uzla zo spájaného zoznamu vyžaduje prehľadanie zoznamu a dealokáciu pamäte. Pri veľkých n rastie čas kvôli cache miss pri dereferenciách ukazovateľov.

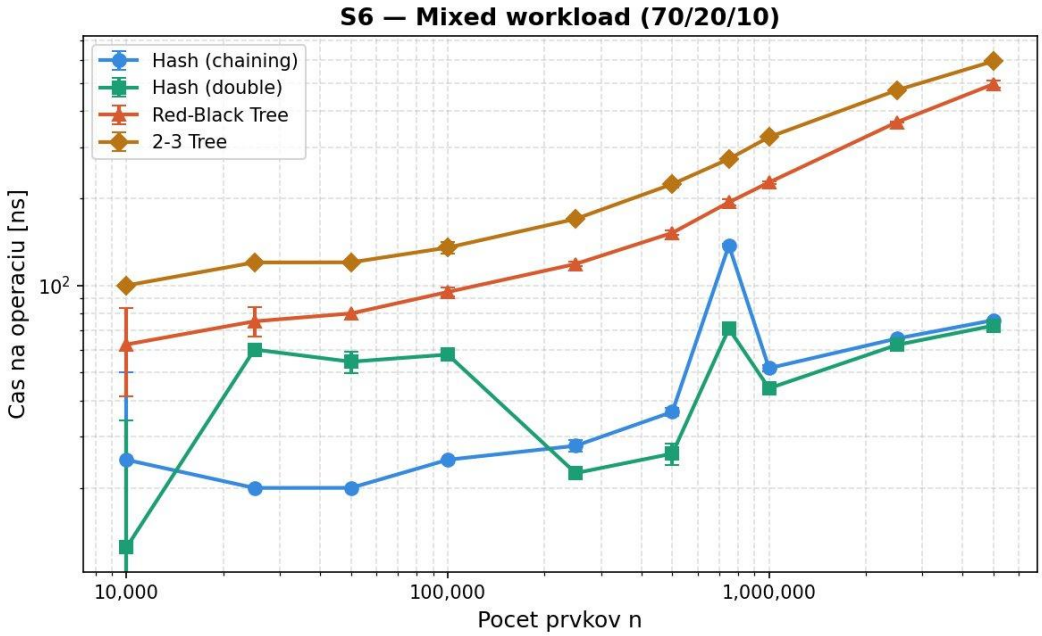
Dôležité: keďže štruktúra sa počas merania vyprázdňuje (ide z n na 0 prvkov), nameraný čas je priemer cez rôzne stavy štruktúry - od plnej po takmer prázdnu. Prvé mazania sú pomalšie (plná štruktúra), posledné rýchlejšie.

Štruktúra	n = 100 000	n = 1 000 000	n = 5 000 000
Hash (reťazenie)	107.4 ns	138.0 ns	203.9 ns
Hash (dvojité)	55.2 ns	88.3 ns	115.7 ns
Red-Black strom	123.8 ns	366.3 ns	773.2 ns
2-3 strom	170.3 ns	430.5 ns	799.1 ns

S6 Zmiešaná záťaž (70 % search / 20 % insert / 10 % delete)

Realistická simulácia pracovného zaťaženia. Štruktúra je pred testom naplnená, operácie sú generované náhodne v pomere 70/20/10.

Setup: vloženie n kľúčov. Meria sa: *n zmiešaných operácií (search existujúcich kľúčov, insert nových, delete existujúcich).*



Graf S6 — Zmiešaná záťaž: čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Keďže 70 % operácií je search hit (kde hash tabuľky dominujú), celkový výsledok je silne ovplyvnený výkonom vyhľadávania. Hash (reťazenie) dosahuje 25–75 ns/op, hash (dvojité rozptýlenie) 12–72 ns/op. Obe sú výrazne lepšie ako stromy.

Pri n = 5 000 000 je hash (reťazenie) ~6.5× rýchlejší ako 2-3 strom (75 vs 593 ns/op). Stromy rastú logaritmicky aj v zmiešanej záťaži, čo potvrdzuje, že ich výkon je dominovaný search operáciami s O(log n) zložitosťou.

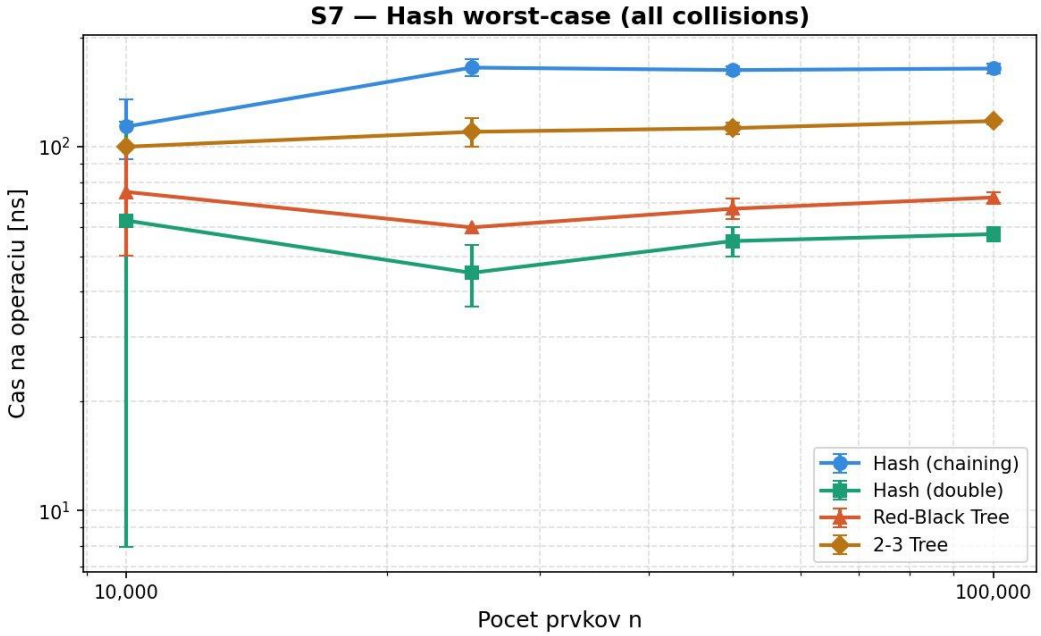
Zaujímavé: hash (reťazenie) je pri S6 rýchlejší ako hash (dvojité rozptýlenie) pri veľkých n. Dôvodom je prevaha search operácií. Reťazenie má pri search hit kratší prístupový čas keď je zoznam krátky (nízka kolízia pri 70% search zaťažení).

Štruktúra	n = 100 000	n = 1 000 000	n = 5 000 000
Hash (reťazenie)	25.0 ns	51.8 ns	75.8 ns
Hash (dvojité)	57.7 ns	44.1 ns	72.6 ns
Red-Black strom	94.9 ns	226.6 ns	495.0 ns
2-3 strom	135.0 ns	324.8 ns	593.9 ns

S7 Najhorší prípad hash tabuľky (všetky kolízie)

Vkladanie kľúčov, ktoré sú násobkami čísla 1009 (stride = 1009). Tieto kľúče spôsobujú maximálne kolízie pri určitých veľkostiach tabuľky. Test je obmedzený na n ≤ 100 000 z dôvodu výkonnosti.

Setup: prázdna štruktúra. Meria sa: *vloženie n kolíznych kľúčov (násobky 1009) vrátane rehashingu.*



Graf S7 — Najhorší prípad hash tabuľky: čas na operáciu [ns] v závislosti od počtu prvkov n

Analýza výsledkov:

Hash (reťazenie) je kolíziami ovplyvnené najviac. Dosahuje 113–164 ns/op. Veľkosť tabuľky sa pri rehashingu počíta ako mocnina 2, čo znamená že násobky 1009 môžu aj po rehashingu padať do rovnakých slotov (ak je nová veľkosť násobkom alebo deliteľom 1009). Rehashing teda kolízie čiastočne rozloží, no nie úplne. Zoznam v kolíznom slote zostáva dlhší ako pri náhodných kľúčoch.

Hash (dvojité) je výrazne odolnejšie: 45–57 ns/op, dokonca rýchlejšie ako pri náhodnom vkladaní. Dôvod: veľkosť tabuľky sa pri rehashingu počíta ako prvočíslo. Prvočíslo nemá spoločných deliteľov so stridou 1009, takže kľúč % veľkosť distribuuje násobky 1009 rovnomerne po celej tabuľke. Kolízny vzor je efektívne eliminovaný. Navyše druhá hash funkcia vytvára iný probe pattern, čo ďalej rozptýli prípadné zvyškové kolízie.

Stromy (RBT a 2-3) nie sú vôbec ovplyvnené kolíznym vstupom — ich výkon závisí len od n, nie od hodnôt kľúčov. Sú dokonca rýchlejšie ako pri náhodnom vkladaní, keďže násobky konštanty tvoria pravidelný vzor ktorý CPU dobre predpovedá.

Štruktúra	n = 10 000	n = 50 000	n = 100 000
Hash (reťazenie)	113.7 ns	162.5 ns	164.1 ns
Hash (dvojité)	62.7 ns	55.0 ns	57.5 ns
Red-Black strom	75.1 ns	67.5 ns	72.5 ns
2-3 strom	100.0 ns	112.5 ns	117.7 ns

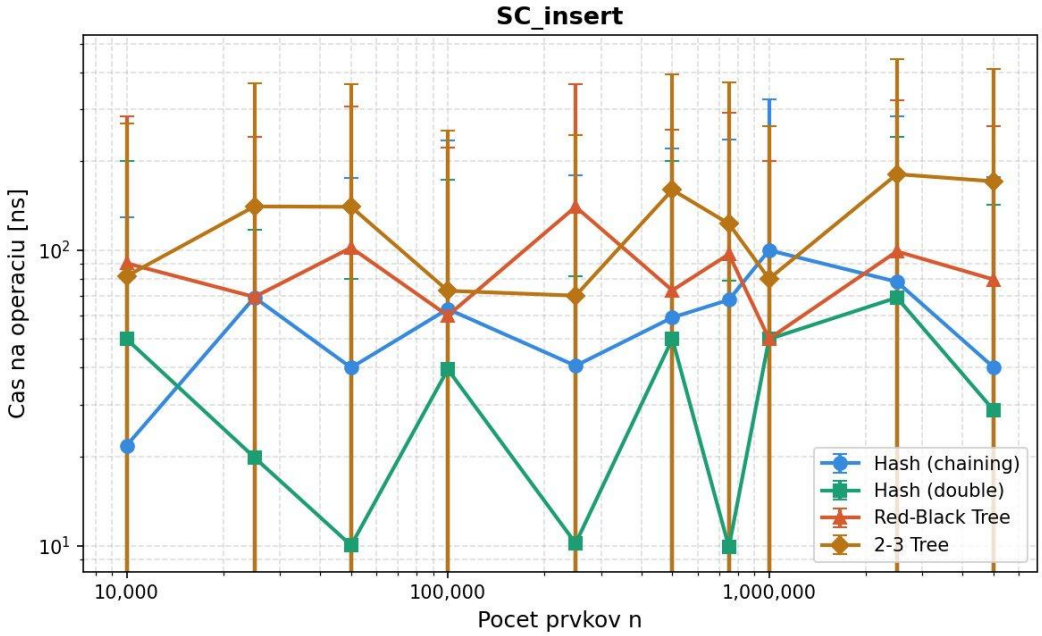
5. Scenáre čistej časovej zložitosti (SC)

SC scenáre merajú čas jednej operácie na štruktúre presnej veľkosti n. Každé meranie pozostáva z bloku 1 000 operácií vydelených 1 000, čím sa prekoná obmedzenie rozlíšenia systémových hodín. Štruktúra zostáva počas merania veľkosti ~n.

Poznámka k presnosti: SC grafy majú výrazne väčšie chybové úsečky ako S grafy. Jednotlivé operácie trvajú 10–200 ns, čo je na hranici rozlíšenia hodín (~100 ns na Windows). Hodnoty treba interpretovať ako orientačné; pre záver o zložitosti je rozhodujúci trend krivky cez rôzne n, nie absolútna hodnota v konkrétnom bode.

SC_insert Vloženie jedného prvku do štruktúry veľkosti n

Setup: vloženie n kľúčov. Meria sa: *blok 1 000 insertov nových záporných kľúčov (štruktúra ide z n na n+1 000).*



Graf SC_insert — Insert: čas jednej operácie [ns] v závislosti od n

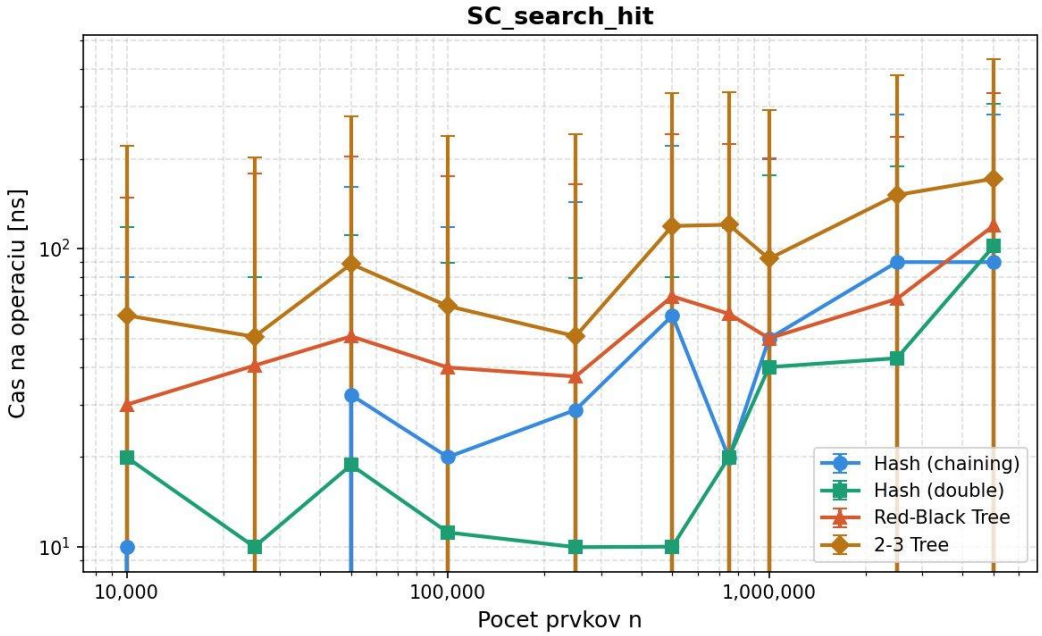
Analýza výsledkov:

Napriek šumu je z grafu zrejmé, že hash tabuľky (reťazenie aj dvojité) vykazujú približne konštantný čas bez výrazného rastového trendu — čo potvrdzuje O(1) amortizovanú zložitosť. Hodnoty oscilujú medzi 10–100 ns bez systematického rastu.

Stromy (RBT, 2-3) majú vyššie hodnoty a mierny rastový trend smerom k väčším n, čo zodpovedá O(log n). 2-3 strom je konzistentne pomalší ako RBT pri inserte (140–170 vs 50–100 ns pri väčších n) - réžia štiepenia uzlov je vyššia.

SC_search_hit Vyhľadanie existujúceho prvku v štruktúre veľkosti n

Setup: vloženie n kľúčov. Meria sa: *blok 1 000 searchov existujúcich kľúčov (štruktúra zostáva presne n).*



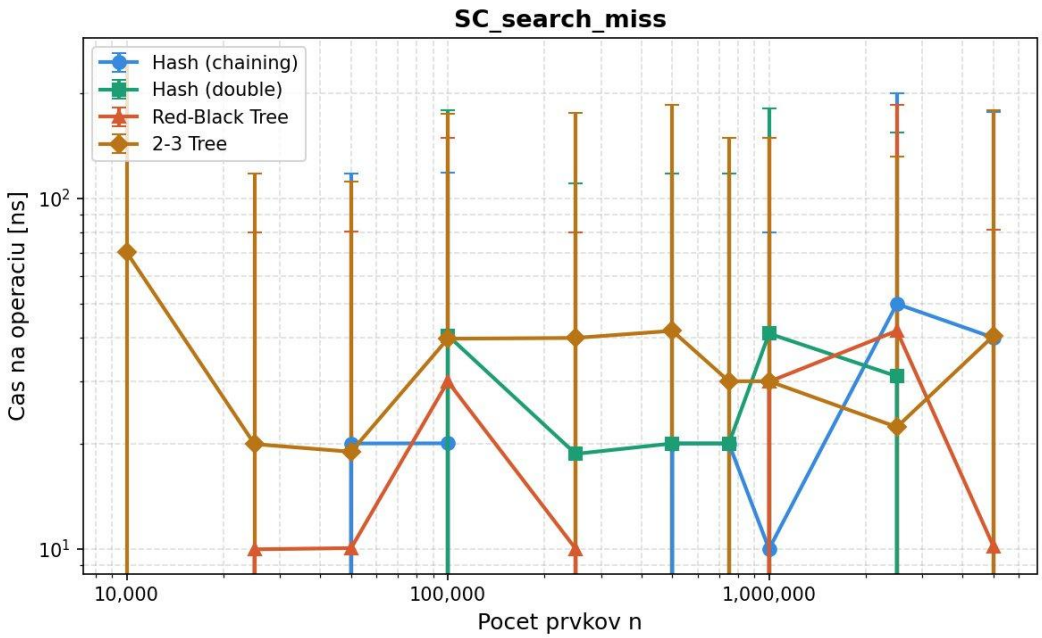
Graf SC_search_hit — Search hit: čas jednej operácie [ns] v závislosti od n

Analýza výsledkov:

Oproti scenáru S3 sú rozdiely medzi štruktúrami tu výrazne menšie. Kým v S3 boli hash tabuľky 5 až 10 násobne rýchlejšie ako stromy, tu sú rozdiely omnoho skromnejšie. Dôvod je pravdepodobne v tom, že S3 meria n searchov naraz a dáta tabuľky sa postupne zahrejú v cache procesora, čo zvýhodňuje hash tabuľky. Tu sa cache zakaždým resetuje pred každým opakovaním.
Hash tabuľky vykazujú pri väčších n rastúci trend napriek teoretickej O(1) zložitosti. Pri väčšom n tabuľka prestane vojsť do cache procesora a prístup do RAM spomaľuje meranie. Stromy rastú miernejšie ako by naznačovala ich O(log n) zložitosť, čo je v súlade s tým, že SC scenáre majú vyšší šum.

SC_search_miss Vyhľadanie neexistujúceho prvku v štruktúre veľkosti n

Setup: vloženie n kladných kľúčov. Meria sa: *blok 1 000 searchov záporných kľúčov (garantovane mimo štruktúry).*



Graf SC_search_miss — Search miss: čas jednej operácie [ns] v závislosti od n

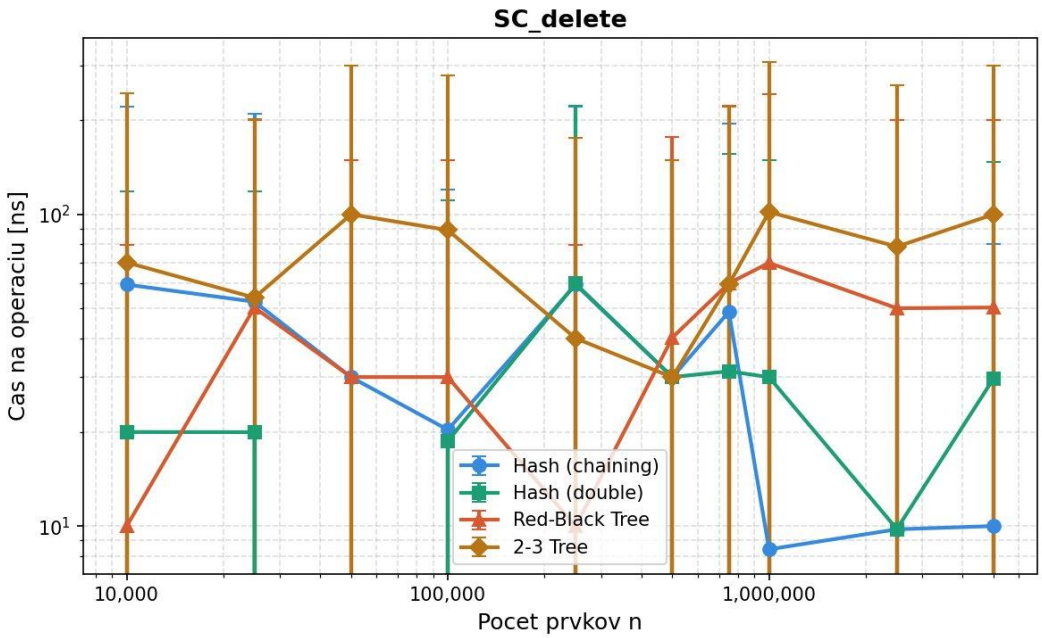
Analýza výsledkov:

Výsledky SC_search_miss sú najviac zašumené zo všetkých SC scenárov. Mnoho meraní vychádza 0 ns (operácia kratšia ako rozlíšenie hodín). Dôvod: záporné kľúče sú okamžite zamietnuté (prázdny slot / ľavý null uzol) bez prehľadávania, čo je rádovo rýchlejšie.

Pre spoľahlivé závery o search miss zložitosti je vhodnejší scenár S4, ktorý meria veľký počet operácií naraz a má výrazne menší šum. Z S4 vieme, že všetky štruktúry vykazujú takmer konštantný čas 10–37 ns pre tento typ miss.

SC_delete Vymazanie prvku zo štruktúry veľkosti n

Setup: vloženie n + 1 000 kľúčov. Meria sa: *blok 1 000 deletov extra kľúčov (štruktúra ide z n+1 000 na n).*



Graf SC_delete — Delete: čas jednej operácie [ns] v závislosti od n

Analýza výsledkov:

Hash tabuľky (reťazenie aj dvojité) vykazujú konštantný čas 10–60 ns bez rastového trendu. To potvrdzuje O(1) amortizovanú zložitosť delete. Tombstone mechanizmus pri dvojitom rozptýlení je obzvlášť efektívny.

Stromy vykazujú mierny rastový trend: RBT ide z ~10 ns pri n = 10 000 na ~50 ns pri n = 5 000 000, 2-3 strom z ~70 na ~100 ns. Nárast zodpovedá O(log n). Šum je výrazný, no trend je štatisticky konzistentný naprieč opakovaní.

6. Záver

Na základe všetkých testovacích scenárov možno zhrnúť nasledujúce závery:

6.1 Porovnanie podľa operácie

Operácia	Najrýchlejšie	Najpomalšie	Pomer (max/min pri n=5M)
Insert (náhodný)	Hash (dvojité)	2-3 strom	~5×
Search (hit)	Hash (reťazenie)	2-3 strom	~10×
Search (miss)	RBT	2-3 strom	~2×
Delete	Hash (dvojité)	2-3 strom	~7×
Zmiešaná záťaž	Hash (reťazenie)	2-3 strom	~8×
Kolízny vstup	Hash (dvojité)	Hash (reťazenie)	~3×

6.2 Potvrdenie teoretickej zložitosti

Štruktúra	Teoretická	Empirická (search hit)	Empirická (insert)
Hash (reťazenie)	O(1) amortiz.	O(1) — mierny rast (cache)	O(1) amortiz.
Hash (dvojité)	O(1) amortiz.	O(1) — stabilný	O(1) amortiz.
Red-Black strom	O(log n)	O(log n) — plynulý rast	O(log n)
2-3 strom	O(log n)	O(log n) — plynulý rast	O(log n)

6.3 Odporúčania pre výber štruktúry

Hash tabuľka s reťazením: Odporúčaná pre aplikácie s dominantným podielom vyhľadávania existujúcich kľúčov (search hit). Najlepšie výsledky v scenároch S3 a S6. Nevhodná pri vstupoch s vysokou kolíznosťou a bez automatického rehashingu.

Hash tabuľka s dvojitým hashovaním: Odporúčaná pre aplikácie s vyváženým mixom insert/delete/search, alebo pri kolíznych vstupoch (S7). Lepšia cache lokalita. Všetky prvky v jednom poli. Konzervatívny prah rehashingu (0.50) zaručuje stabilný výkon.

Červeno-čierny strom: Odporúčaný keď je potrebné usporiadanie prvkov (in-order traversal, rozsahové dopyty, predecessor/successor). Garantuje O(log n) aj pre najhorší vstup. Pre čisté vyhľadávanie a mazanie je 5–10× pomalší ako hash tabuľky pri veľkých n.

2-3 strom: Poskytuje dokonalé vyváženie a O(log n) zložitosť. Napriek nižšej hĺbke stromu je v praxi pomalší ako RBT. Vyššia réžia spracovania uzlov s dvoma kľúčmi prevažuje nad výhodou nižšej hĺbky. Vhodný pre systémy kde je garantované vyváženie kritickejšie ako absolútna rýchlosť (napr. B-strom varianty v databázach).